

Recovery-Enhanced Imitation Learning for Robust Multi-Task Robot Manipulation

[[TODO: replace with verified author names]]
[[TODO: replace with affiliations and contact email]]

Abstract—Action-chunking policies can execute long, smooth manipulation sequences, but a small state or action deviation may move the robot outside the distribution represented by the current chunk. We present Recovery-Enhanced Imitation Learning (REIM), a runtime wrapper that combines a task-conditioned Action Chunking with Transformers policy, a causal LSTM risk monitor, and a task-conditioned recovery policy through hysteretic control arbitration. The monitor is trained on future behavioral-deviation events, while recovery supervision is collected from successful expert continuations at online trigger states. We evaluate separately trained models on the known-task, goal-observable Meta-World MT10 and MT50 suites. The clean protocol and a paired Gaussian-perturbation extension each use 50 episodes per task. At the strongest tested perturbation, REIM attains 63.0% task-macro success on MT10 and 49.52% on MT50, compared with 46.4% and 33.96% for the same learned recovery policy under a reward-stagnation gate. Clean success remains 97.6% and 92.24%, respectively. Paired outcomes and per-task analysis show that recovery is beneficial on most, but not all, task families; high intervention occupancy under strong perturbations remains an important limitation.

Index Terms—robot learning, imitation learning, recovery policy, runtime monitoring, multi-task manipulation

I. Introduction

Imitation learning can produce effective manipulation controllers without an explicit dynamics model, yet closed-loop deployment remains sensitive to covariate shift. A small execution error changes the next observation, which changes the next prediction, and the resulting state sequence may progressively depart from the demonstrations. Action chunking reduces high-frequency action inconsistency by predicting temporally extended commands, but it does not by itself decide when the nominal policy should stop controlling the robot. This distinction matters in long-horizon manipulation: continuing a locally inappropriate chunk can convert a recoverable deviation into a task failure.

Existing recovery systems often assume either a hand-designed trigger, an external safety signal, or a separate reinforcement-learning objective. These choices complicate a fair diagnosis of where improvement comes from. A recovery policy may be competent while its trigger is late; conversely, an aggressive trigger may invoke recovery almost continuously and harm nominal states. The central problem is therefore not only how to learn recovery actions, but how to arbitrate between nominal and recovery control using causal information available at runtime.

We introduce Recovery-Enhanced Imitation Learning (REIM), a supervised recovery architecture for known-task multi-task manipulation. As illustrated in Fig. 1, a task-conditioned ACT policy supplies nominal action chunks, an LSTM consumes the most recent state history and estimates near-term behavioral-deviation risk, and a task-conditioned multilayer perceptron supplies recovery actions. A hysteretic state machine turns the continuous risk estimate into stable control decisions. Separate entry and release conditions prevent rapid switching, a minimum dwell time gives recovery an opportunity to act, and a cooldown period prevents immediate retriggering.

The risk target is not terminal task success. Instead, it identifies whether a task-calibrated disagreement event will occur in the immediate future. Consequently, detector precision and end-to-end task success are different quantities: a monitor with moderate classification precision can still be useful when its mistakes are filtered by hysteresis and the downstream policy is trained on the states where intervention actually occurs. We evaluate this system on MT10 and MT50 with task identity exposed, following the goal-observable use of these suites. The two suites share the same disturbance definition and evaluation logic but use separately trained models; their absolute scores are therefore compared as evidence of scaling behavior, not as samples from one pooled policy.

This work makes three contributions:

- a causal, task-conditioned risk monitor with validation-only operating-point selection and explicit hysteretic arbitration;
- a trigger-aligned supervised recovery dataset constructed from successful expert continuations at policy-induced states; and
- a paired evaluation across MT10 and MT50 that reports clean performance, a common perturbation sweep, rescued and harmed episodes, and per-task heterogeneity.

We deliberately avoid a state-of-the-art claim. The evidence supports robustness under the tested simulation protocol; it does not establish unseen-task generalization, calibrated failure probabilities, or real-robot reliability.

II. Related Work

A. Imitation Learning and Action Chunking

Behavioral cloning directly fits expert state-action pairs but can accumulate errors when the learned policy visits

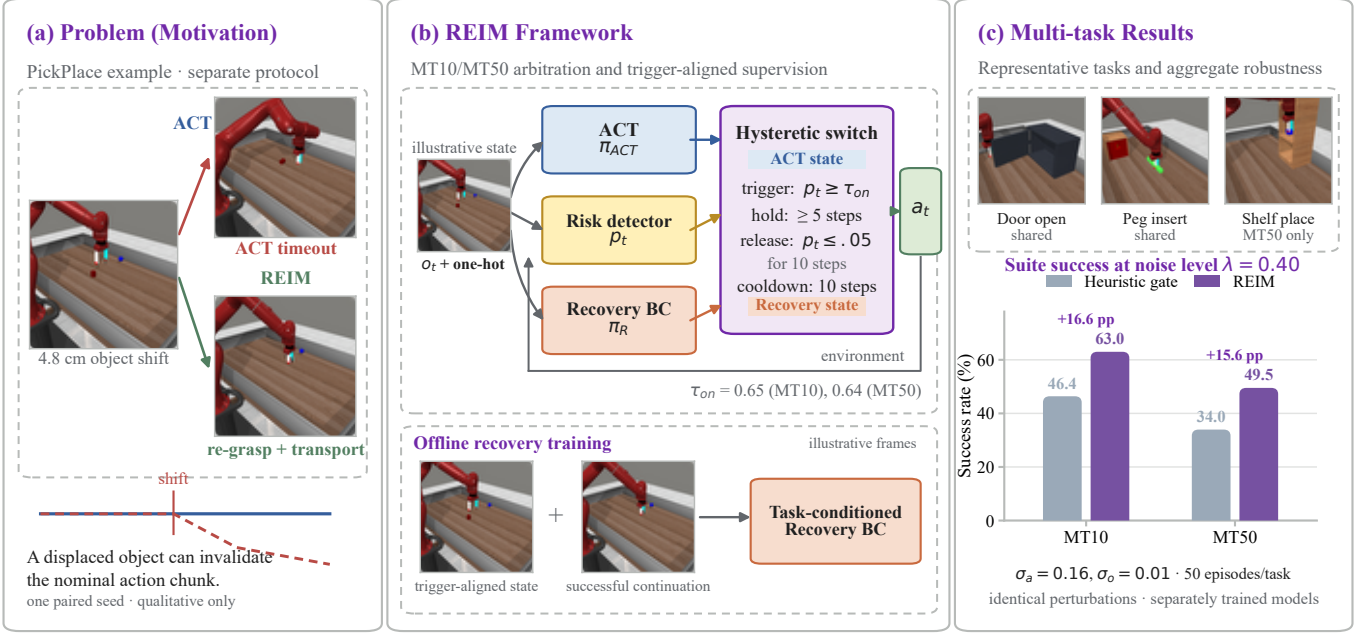


Fig. 1. REIM overview. (a) A separate PickPlace example motivates recovery after ACT drift. (b) Task-conditioned nominal control, causal risk monitoring, and hysteretic arbitration. (c) Representative MT tasks and headline results at $\lambda = 0.40$; MT10 and MT50 models are trained separately.

states absent from the demonstrations. Dataset aggregation addresses this mismatch by querying an expert at states induced by the learner [1]. ACT instead predicts fixed-length action chunks with a conditional variational Transformer and temporally ensembles overlapping predictions [2]. REIM uses ACT as the nominal controller. Its recovery collection is related to dataset aggregation because supervision is obtained at policy-induced trigger states, although the present pipeline does not iteratively retrain the nominal policy.

B. Runtime Monitoring and Recovery

Runtime monitors can detect inconsistency or stalled progress without changing the base policy [3]. Learned recovery has also been studied through safe reinforcement learning, residual control, meta-reasoning, and local policy chaining [4]–[7]. REIM differs in scope: both the recovery policy and the risk monitor are trained by supervised learning, and the learned gate is compared with a reward-stagnation gate while holding the recovery policy fixed. This isolates the contribution of arbitration from the existence of a recovery controller.

III. Method

A. Problem Formulation

Consider a manipulation suite with K known tasks. At step t , the environment provides a 39-dimensional state observation o_t and task identity k . We concatenate the state with a one-hot task code,

$$x_t = [o_t; e_k] \in \mathbb{R}^{39+K}, \quad (1)$$

so MT10 and MT50 use 49- and 89-dimensional policy inputs. An episode succeeds when the environment’s official success signal is observed within 500 steps. REIM combines a nominal policy π_N , risk monitor D , recovery policy π_R , and arbitration state $z_t \in \{N, R\}$.

B. Nominal Multi-Task ACT

The nominal controller is a state-based ACT policy [2]. Given x_t , it predicts a chunk of 20 four-dimensional actions. Training minimizes an ℓ_1 reconstruction loss plus the ACT latent KL term. Overlapping predictions are combined using exponential temporal ensembling. We train distinct ACT models for MT10 and MT50 because the task vocabularies and input dimensions differ. Both use three encoder layers, four decoder layers, eight attention heads, and KL weight 10; hidden/latent dimensions are 256/32 for MT10 and 384/48 for MT50.

C. Causal Near-Term Risk Monitor

The monitor receives only the latest $L = 10$ conditioned states. A one-layer LSTM maps $x_{t-L+1:t}$ to a hidden representation, followed by a two-layer classifier and sigmoid output

$$p_t = \sigma(D(x_{t-L+1:t})). \quad (2)$$

Because no future observation is used at inference, the monitor is causal.

Risk labels are derived from action disagreement rather than terminal success alone. For each task, the 90th percentile of ℓ_1 disagreement between ACT and the scripted expert is fitted on the failure-training bank. A step is a

risk event when its disagreement reaches that task-specific threshold. Failed rollouts also mark their final 25 steps as events. The binary target at time t is positive if an event occurs within the next 10 steps. Task thresholds fitted on the training bank are frozen for validation and final evaluation.

The detector is optimized with weighted binary cross entropy. Deployment thresholds are selected only on the independent validation bank subject to task-macro precision of at least 0.60; the selected operating points are $\tau_{\text{on}} = 0.65$ for MT10 and 0.64 for MT50. These validation metrics are not task success rates and should not be read as such.

D. Trigger-Aligned Recovery Policy

Recovery data are collected by running the nominal ACT policy and monitoring risk online. A deliberately broad collection threshold of 0.20 captures candidate recovery starts. Once triggered, a scripted task expert controls the continuation. Only successful continuations are retained, yielding 50 trigger-aligned trajectories per task. This creates recovery supervision at states encountered by the learned nominal controller rather than only at states from clean demonstrations.

The recovery controller is a deterministic task-conditioned MLP with hidden dimensions 512–512–256 and Tanh activations. It is trained with Smooth- ℓ_1 action loss, state noise augmentation of standard deviation 0.005, and task-balanced sampling. MT10 contains 500 recovery trajectories (30,807 training and 7,973 validation steps); MT50 contains 2,500 trajectories (159,581 training and 39,746 validation steps). As with ACT, a separate recovery model is trained for each suite.

E. Hysteretic Control Arbitration

The controller enters recovery when $p_t \geq \tau_{\text{on}}$ and no cooldown is active. It remains in recovery for at least five steps. Thereafter, it releases control only after $p_t \leq 0.05$ for ten consecutive steps. A ten-step cooldown follows release, and a single intervention is capped at 250 recovery steps. Formally, the principal transitions are

$$z_t = N \rightarrow R \quad \text{if } p_t \geq \tau_{\text{on}} \text{ and } \text{cooldown} = 0, \quad (3)$$

$$z_t = R \rightarrow N \quad \text{if } n_R \geq 5 \text{ and } n_{\text{safe}} \geq 10. \quad (4)$$

The thresholds implement hysteresis: evidence required to enter recovery is intentionally much stronger than evidence required to remain nominal after a sustained safe period. ACT’s temporal state is reset whenever control returns from recovery.

F. Why Collection and Deployment Thresholds Differ

The recovery collection threshold and deployment threshold serve different statistical purposes. During collection, the objective is coverage: the recovery learner should see a broad range of states near the boundary of nominal behavior. A lower threshold of 0.20 therefore

favours recall and lets the expert demonstrate how to continue before a deviation becomes terminal. At deployment, the objective is control selection: an unnecessary switch can interrupt a successful ACT chunk. The higher suite-specific thresholds are consequently chosen under a precision constraint. This separation is fixed before final evaluation and should not be interpreted as tuning the recovery policy on test outcomes.

The same distinction explains why the monitor is trained on near-term deviation rather than episode-level failure. A terminal label assigns one outcome to hundreds of preceding states, many of which may be nominal, and gives little temporal guidance about when to switch. The event target localizes supervision around task-relative disagreement and propagates it only ten steps backward. The resulting score is an operational ranking signal. Hysteresis then converts that signal into a decision using temporal persistence rather than assuming that each score is a calibrated failure probability.

IV. Experimental Protocol

A. Benchmarks and Data Separation

We use Meta-World 3.1.1 [8] with MuJoCo [9]. MT10 and MT50 provide 10 and 50 manipulation task families with known task identities. We use the goal-observable state representation and the official 500-step horizon. For each suite, task variants are reserved in distinct banks for demonstrations, detector training, recovery training, detector validation, and final evaluation. A digest-based audit verifies that the materialized task payloads and sampling-unit identities are disjoint across these stages.

Each task contributes 50 expert demonstrations, 50 failure-training rollouts, 50 retained recovery continuations, and 20 detector-validation rollouts. Final results use a separate confirmation bank with 50 episodes per task: 500 paired episodes per method for MT10 and 2,500 for MT50. Task variants, episode seeds, and injected noise arrays are shared across methods within a condition. No data from the confirmation bank are used to train a model or select a threshold.

Table I summarizes the scale of the two independently trained suites. The counts are deliberately symmetric per task; MT50 increases coverage by adding task families rather than by allocating fewer samples to each family. This design makes the MT10/MT50 comparison informative about breadth while avoiding a sample-budget confound at the task level.

B. Methods Compared

We compare four controllers. MT-MLP BC is a two-layer 256–256 behavioral-cloning policy. MT-ACT is the nominal action-chunking controller. Heuristic+recovery uses the same learned recovery policy as REIM but triggers when reward variation over a 20-step window falls below 0.01 after step 30. REIM replaces this heuristic with the learned risk monitor and hysteretic state machine.

TABLE I
Suite construction and final evaluation scale.

Quantity	MT10	MT50
Task families	10	50
Conditioned state dimension	49	89
Expert demonstrations	500	2,500
Failure-training rollouts	500	2,500
Recovery continuations	500	2,500
Detector-validation rollouts	200	1,000
Episodes/method/condition	500	2,500
Final task-macro units	10	50

TABLE II
Validated detector operating points and frozen control settings.

Quantity	MT10	MT50
Trigger threshold	0.65	0.64
Task-macro precision	0.600	0.600
Task-macro recall	0.505	0.535
Task-macro ECE	0.138	0.141
Release threshold	0.05	0.05
Release patience (steps)	10	10
Minimum recovery (steps)	5	5
Cooldown (steps)	10	10
Recovery budget (steps)	250	250

Holding π_R fixed between the last two methods makes their difference an arbitration comparison.

Table II summarizes the validation-selected detector operating points and the controller settings frozen before confirmation evaluation. The precision, recall, and expected calibration error (ECE) entries characterize near-term risk-event classification on the validation bank; they are not task-success metrics.

C. Clean and Perturbed Evaluation

Official-clean evaluation uses unmodified benchmark execution. Robustness is evaluated separately with task-uniform independent Gaussian noise. For level λ ,

$$\epsilon_t^a \sim \mathcal{N}(0, (0.4\lambda)^2 I_4), \quad \epsilon_t^o \sim \mathcal{N}(0, (0.025\lambda)^2 I_{39}), \quad (5)$$

with $\lambda \in \{0, 0.10, 0.20, 0.30, 0.40\}$. Actions are clipped to the environment range after perturbation. Object positions are never teleported in this multi-task protocol. At $\lambda = 0.40$, the action and observation standard deviations are 0.16 and 0.01. This sweep is a REIM robustness extension, not an official Meta-World score.

The primary metric is task-macro success: success is averaged within each task and then equally across tasks. We report 95% within-task episode-bootstrap intervals using 5,000 bootstrap samples. Paired rescued and harmed counts use the same task variant, episode seed, and noise realization for the two compared methods. Recovery occupancy is the fraction of executed steps controlled by π_R .

For paired comparison against the heuristic gate, let s_i^R and s_i^H be binary outcomes for matched episode i . We report rescued episodes when $(s_i^R, s_i^H) = (1, 0)$ and

harmed episodes when $(s_i^R, s_i^H) = (0, 1)$. Their normalized difference equals the paired success-rate change,

$$\Delta_{R-H} = \frac{1}{N} \sum_{i=1}^N (s_i^R - s_i^H) = \frac{N_{\text{rescued}} - N_{\text{harmed}}}{N}. \quad (6)$$

This decomposition retains the direction of every discordant episode and prevents an aggregate improvement from being mistaken for uniform benefit.

V. Results

A. Clean Performance Is Largely Preserved

At $\lambda = 0$, MT-ACT obtains 97.4% success on MT10 and 91.64% on MT50. REIM obtains 97.6% [96.2, 98.8] and 92.24% [91.4, 93.04], respectively. Thus adding the learned gate and recovery controller changes clean success by only +0.2 and +0.6 percentage points relative to ACT. The heuristic gate is slightly higher on clean execution—98.0% on MT10 and 94.0% on MT50—so the data do not support a claim that REIM is the best clean controller. REIM still intervenes in 46.0% of MT10 and 28.64% of MT50 clean episodes, with mean recovery occupancies of 18.70% and 16.83%; the detector is therefore not inactive on nominal rollouts.

B. A Common Perturbation Sweep Links MT10 and MT50

Figure 2 places MT10 and MT50 under the same perturbation definition and evaluation budget. The trend is consistent across suites. At $\lambda = 0.10$, the heuristic gate is modestly better: 49.8% versus 45.4% on MT10 and 35.84% versus 34.88% on MT50. At $\lambda = 0.20$, REIM reaches 53.0% and 41.28%, compared with 50.6% and 35.96% for the heuristic. The gap then increases. At $\lambda = 0.30$, the respective REIM advantages are 9.4 and 11.88 points; at $\lambda = 0.40$, they are 16.6 and 15.56 points.

MT-ACT and MT-MLP BC fall to single-digit success once perturbations are introduced. At $\lambda = 0.40$, ACT reaches 6.0% on MT10 and 2.04% on MT50. This sharp degradation indicates that the tested noise moves execution outside the tolerance of both nominal policies. The recovery comparisons are therefore the informative part of the high-noise regime.

C. Strong Perturbations: Net Gains with Two-Sided Effects

Figure 3 expands the strongest tested condition. REIM reaches 63.0% [60.2, 65.6] on MT10 and 49.52% [48.16, 50.92] on MT50. The heuristic gate reaches 46.4% [43.2, 49.8] and 33.96% [32.52, 35.44]. The common recovery policy is therefore not sufficient to explain the difference; how recovery is invoked matters.

The paired decomposition reveals both benefit and harm. On MT10, REIM rescues 115 of 500 episodes (23.0%) that the heuristic loses, while harming 32 (6.4%) that the heuristic solves, for a net gain of 16.6 points. On MT50, it rescues 610 of 2,500 episodes (24.4%)

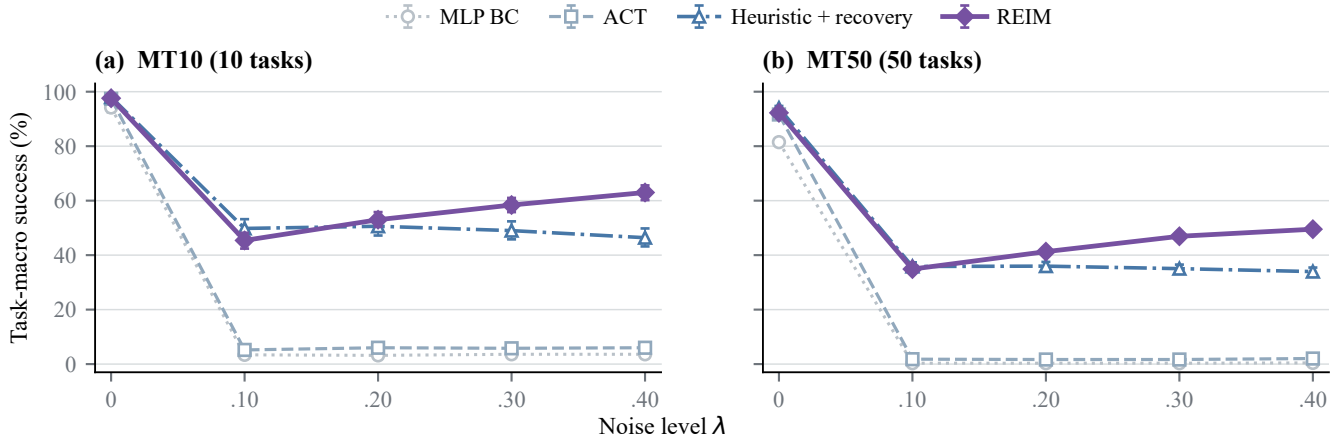


Fig. 2. Task-macro success across the shared MT10/MT50 perturbation sweep. Error bars are 95% within-task bootstrap intervals over 50 episodes per task. Perturbed conditions are a non-official robustness extension.

and harms 221 (8.84%), for a net gain of 15.56 points. Reporting only aggregate success would hide these adverse switches. It would also hide control cost: at $\lambda = 0.40$, REIM intervenes in 93.4% of MT10 and 98.68% of MT50 episodes, with mean recovery occupancies of 69.21% and 67.40%, respectively.

D. Benefits Vary Across Task Families

Figure 4 compares the learned and heuristic gates per task. REIM improves 7 of 10 MT10 tasks and 32 of 50 MT50 tasks, ties on 3 MT50 tasks, and is worse on the remaining 3 and 15 tasks. Both shared and MT50-specific task families appear on either side of the diagonal. The improvement therefore does not arise from one small subset of tasks, but neither is it universal.

The largest positive differences are highly structured. Door-open improves by 82 points in both suites; button-press-topdown improves by 34 points on MT10 and 72 on MT50; and window-close improves by 34 and 70 points. Conversely, MT10 drawer-close and push decline by 14 and 10 points, while MT50 stick-push, coffee-pull, and coffee-push decline by 50, 28, and 26 points. The repeated gains on task families present in both suites support the cross-suite consistency of the learned trigger. The large negative MT50 cases also show why a suite mean cannot substitute for task-level reporting.

The negative cases are consistent with two failure modes. First, a moderate-precision detector may trigger recovery in a state where the nominal chunk would have completed the task. Second, the task-conditioned recovery MLP may not represent every contact mode equally well, especially when the perturbation changes the state faster than the successful continuation data. These interpretations are hypotheses supported by the paired pattern, not separately identified causal effects.

E. Component Training-Seed Stability

We additionally vary the detector and recovery optimization seed while keeping the ACT checkpoint fixed. The MT10 study uses seeds 42–44 on its independent 50-episode-per-task bank. The MT50 study uses seeds 42–45 on the main confirmation bank; its recovery-collection bank and deployment threshold are fixed. For each MT50 seed, REIM and the heuristic gate use the same seed-specific recovery checkpoint, so their difference isolates arbitration at deployment. Table III reports means and sample standard deviations without pooling the two study banks. Under $\lambda = 0.40$, all four MT50 REIM seeds reach 47.24–49.52% success, while their matched heuristic controllers reach 33.96–36.00%; the paired advantage is 12.52–15.56 points (mean 13.65). At $\lambda = 0.10$, however, MT50 seed variation is larger and every REIM seed remains below its matched heuristic, reinforcing that the learned gate is not advantageous at the lightest perturbation.

VI. Discussion and Limitations

The experiments show that learned risk arbitration is most useful when perturbations push nominal policies far outside their training distribution. REIM increasingly outperforms reward-stagnation gating as disturbance strength grows, and this pattern holds across MT10 and MT50 and across component-training seeds. The detector’s value

TABLE III
Component training-seed stability: task-macro success (%).

Suite	λ	REIM	Heuristic
MT10 (3 seeds)	0	96.6 $\pm$ 0.4	96.7 $\pm$ 0.9
	0.10	44.1 $\pm$ 1.3	44.7 $\pm$ 3.2
	0.40	60.8 $\pm$ 4.9	40.8 $\pm$ 3.6
MT50 (4 seeds)	0	92.36 $\pm$ 0.35	93.46 $\pm$ 0.62
	0.10	34.39 $\pm$ 2.59	37.38 $\pm$ 1.31
	0.40	48.75 $\pm$ 1.05	35.10 $\pm$ 0.94

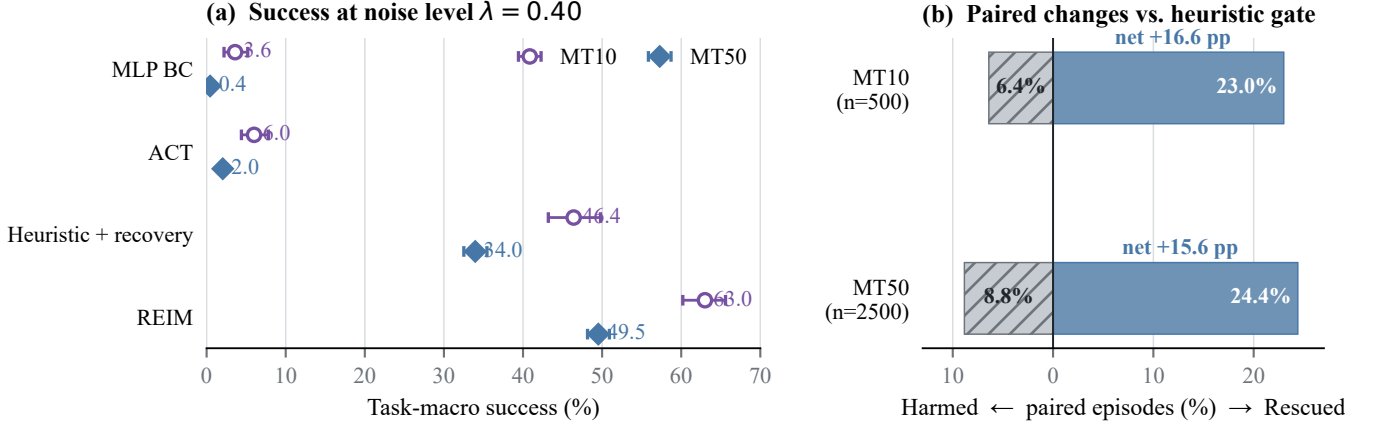


Fig. 3. Results at $\lambda = 0.40$. (a) Task-macro success with 95% bootstrap intervals. (b) Paired REIM outcomes relative to heuristic-gated recovery; rescued and harmed are normalized by the suite episode count.

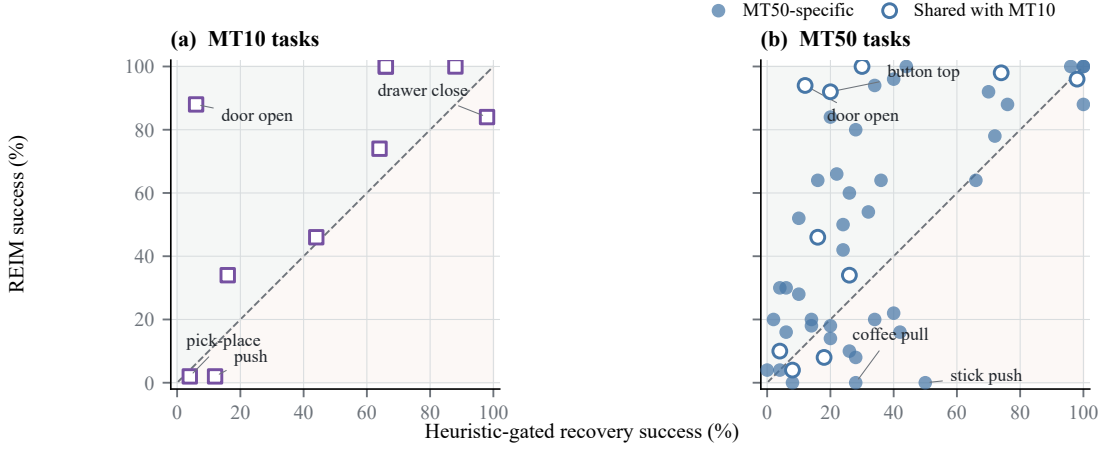


Fig. 4. Per-task REIM versus heuristic-gated recovery at $\lambda = 0.40$. Each point is a 50-episode task mean; points above the diagonal favor REIM. Hollow MT50 markers denote task families shared with MT10.

lies in coupling early risk estimates to a recovery policy through temporally constrained arbitration rather than treating risk classification as an endpoint. Hysteresis turns the score into stable control decisions, while paired analysis identifies where mistimed interventions still reduce performance. The central opportunity is therefore to retain the strong-noise gains while lowering unnecessary recovery occupancy.

The present evidence is limited to known-task Meta-World simulation, separately trained MT10 and MT50 models, and Gaussian state-action perturbations. It does not establish unseen-task or real-robot transfer, and successful expert continuations may not cover every contact-rich recovery state. Structured disturbances, occupancy-matched comparisons, and physical-robot evaluation are the most direct extensions of this study.

VII. Conclusion

REIM combines action-chunking imitation, causal near-term risk monitoring, supervised recovery, and hysteretic arbitration in one multi-task manipulation controller. Across matched MT10 and MT50 perturbation sweeps, it preserves clean performance and improves strong-noise success over both nominal policies and a heuristic gate using the same recovery controller. Paired and per-task analyses show why an aggregate gain is not the whole story: interventions both rescue and harm episodes, and their effect varies by task. The next empirical step is to reduce unnecessary intervention while testing structured disturbances and real-robot execution.

References

- [1] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. *Proceedings of Machine Learning Research*, vol. 15. PMLR, 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html>
- [2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, Jul. 2023.
- [3] C. Agia, R. Sinha, J. Yang, Z. Cao, R. Antonova, M. Pavone, and J. Bohg, “Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress,” in *Proceedings of the 8th Conference on Robot Learning*, ser. *Proceedings of Machine Learning Research*, vol. 270. PMLR, 2025, pp. 689–723. [Online]. Available: <https://proceedings.mlr.press/v270/agia25a.html>
- [4] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, “Recovery RL: Safe reinforcement learning with learned recovery zones,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4915–4922, 2021.
- [5] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *2019 International Conference on Robotics and Automation*, 2019, pp. 6023–6029.
- [6] S. Vats, M. Likhachev, and O. Kroemer, “Efficient recovery learning using model predictive meta-reasoning,” in *2023 IEEE International Conference on Robotics and Automation*, 2023, pp. 7258–7264.
- [7] S. Vats, D. K. Jha, M. Likhachev, O. Kroemer, and D. Romeres, “Recoverychaining: Learning local recovery policies for robust manipulation,” *arXiv preprint arXiv:2410.13979*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.13979>
- [8] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning,” in *Proceedings of the Conference on Robot Learning*, ser. *Proceedings of Machine Learning Research*, vol. 100. PMLR, 2020, pp. 1094–1100. [Online]. Available: <https://proceedings.mlr.press/v100/yu20a.html>
- [9] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.